

Exploring Sequential Dynamics on Temporal Graphs via Composite Filtering

Yuanyuan Xu University of New South Wales Sydney, Australia yuanyuan.xu@unsw.edu.au	Danni Wu East China Normal University Shanghai, China danni.wu@stu.ecnu.edu.cn	Xuemin Lin The Chinese University of Hong Kong, Shenzhen Shenzhen, China xuemin.lin@gmail.com	Dong Wen University of New South Wales Sydney, Australia dong.wen@unsw.edu.au
---	--	---	---

Wenjie Zhang University of New South Wales Sydney, Australia wenjie.zhang@unsw.edu.au	Lei Chen The Hong Kong University of Science and Technology (Guangzhou) Guangzhou, China The Hong Kong University of Science and Technology Hong Kong, Hong Kong leichen@cse.ust.hk	Ying Zhang* Laboratory for Statistical Monitoring and Intelligent Governance of Common Prosperity Zhejiang GongShang University Hangzhou, China ying.zhang@zjgsu.edu.cn
---	---	---

Abstract

Real-world temporal graphs are largely driven by sequential dynamics, and edge repetitions are rare. This characteristic has spotlighted a key limitation of existing temporal graph neural networks (T-GNNs): on such graphs, state-of-the-art T-GNNs often achieve less than 70% MRR on link prediction. Two factors drive this short-fall: (1) Memory modules and neighbor co-occurrence encodings in existing T-GNNs often fail since they rely on memorizing exact neighbor identities and on the co-occurrence assumption. (2) Existing T-GNNs are sensitive to abrupt events, which are common in sequential settings that can exceed ten million updates, thereby compromising generalization. To tackle the challenges of sequential dynamics, we propose SeqFilter, a simple yet robust neural network that functions as a composite filter for link prediction on temporal graphs. SeqFilter comprises two modules: a node rhythm memory and a frequency-selective structure encoder. The node rhythm memory shifts the focus from *who* interacts to *when*, modeling absolute timestamps with recency awareness to capture each node's interaction rhythm. To model temporal structures in complex sequential dynamics, we propose a frequency-selective structure encoder that amplifies or suppresses specific frequencies in the neighbor spectrum, enabling the effective modeling of local structure correlations. Theoretically, this encoder functions as a cascade of three learnable filters that approximate the optimal linear denoiser, helping capture the underlying structural patterns. Last, SeqFilter fuses the outputs of two modules to generate high-quality node embeddings. Extensive experiments across eight sequential dynamic datasets show

that SeqFilter outperforms 11 baselines by an average improvement of 15.82% in MRR while achieving an order of magnitude speedup compared to the frequency-enhanced baseline.

CCS Concepts

• Computing methodologies → Machine learning.

Keywords

Temporal graph; sequential dynamics; composite filter

ACM Reference Format:

Yuanyuan Xu, Danni Wu, Xuemin Lin, Dong Wen, Wenjie Zhang, Lei Chen, and Ying Zhang. 2026. Exploring Sequential Dynamics on Temporal Graphs via Composite Filtering. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792101>

1 Introduction

Temporal link prediction is a fundamental task in dynamic systems such as social networks [2, 5, 10], e-commerce platforms [4, 26, 45], and biometric networks [12, 17]. Recent studies [54] reveal that these systems often exhibit complex *sequential dynamics*. For example, on Amazon, a customer who has just ordered a treadmill is more likely to buy running shoes or knee pads next, rather than ordering another treadmill. Such sequential behaviors are primarily driven by temporal ordering rather than behavioral recurrence, as the TGB-Seq benchmark [54] statistics show six of the eight datasets report a repeat ratio close to 0%. Consequently, the temporal link prediction shifts towards predicting *unseen* interactions by modeling complex sequential dynamics from historical interactions.

T-GNNs [6, 9, 18, 23, 27, 34, 37, 42, 43, 48, 50, 57] have achieved promising results for link prediction on temporal graphs rich in repeated interactions. However, evaluation on the latest TGB-Seq benchmark [54], whose datasets feature sequential dynamics, reveals a significant performance drop: mainstream T-GNNs typically

*Corresponding author.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.
WWW '26, Dubai, United Arab Emirates
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792101>

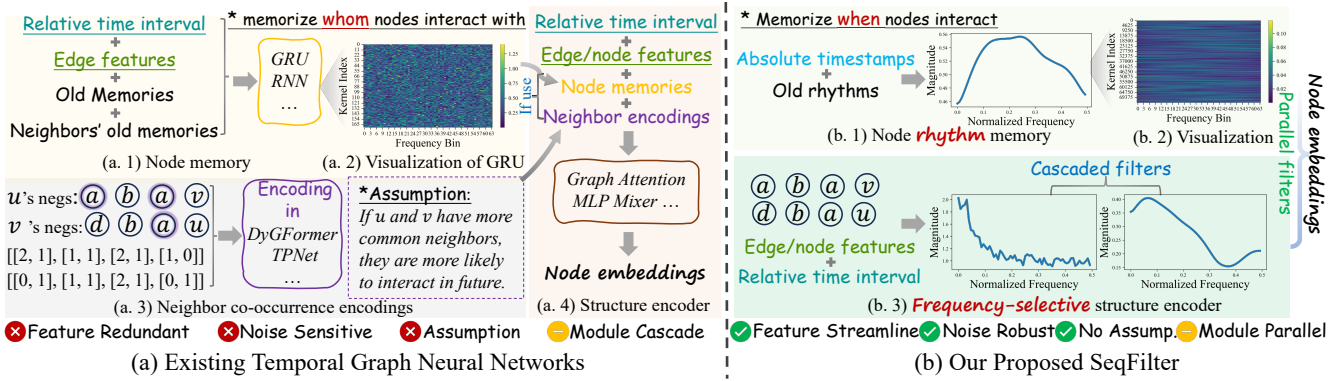


Figure 1: A comparison between mainstream T-GNNs and our SeqFilter. For T-GNNs, we use TGN [27] as the memory module and DyGFormer [57] for neighbor co-occurrence encodings as illustrations. For SeqFilter, we visualize learned weights for modules using the YouTube dataset.

achieve MRR scores below 70% and suffer up to an *eight-fold* performance decline when transitioning from predicting historical interactions to forecasting unseen interactions. The TGB-Seq benchmark traces these shortcomings to their structural functionality. Beyond its analysis, we provide the following observations.

Observations. This work identifies four key reasons why existing T-GNNs struggle to model sequential dynamics via theoretical and empirical analyses. First, the memory modules in many T-GNNs [18, 27, 37, 42, 48] are designed to memorize nodes’ *exact* historical neighbor features (e.g., edge features and relative time intervals) using recurrent units. However, as illustrated in Fig. 1a.2, a frequency-domain visualization¹ of GRU weights of the TGN [27] on the YouTube dataset reveals a flat and unstructured spectrum, suggesting that this *who-based* memorization hardly captures meaningful temporal patterns in sequential dynamic scenarios. Second, several T-GNNs [27, 37, 48] feed both node memories (which already store neighbor features) and neighbor features into the structure encoder, leading to redundant information that exacerbates overfitting and weakens generalization. Third, recent explorations in neighbor co-occurrence encodings [6, 23, 29, 34, 43, 57] rely on the co-occurrence assumption that nodes sharing more historical neighbors are more likely to interact, an effectiveness assumption for temporal graphs with repeated edges (Fig. 1a.3). However, this assumption does not perform well in sequential dynamic graphs, where shared neighbors may be rare and future interactions rely mainly on recent event order, not on long-term shared neighbors. This also explains why complex T-GNNs like DyGFormer [57] often underperform simpler ones such as GraphMixer [9] on TGB-Seq datasets [54]. Finally, in sequential dynamic modeling, existing T-GNNs regard abrupt or outlier interactions, which are common in frequently evolving graphs (e.g., over 10 million update on Yelp), as routine events. This distorts temporal structure learning, compromising effectiveness and generalization.

Inspired by the above observations, we investigate complex sequential dynamic modeling on temporal graphs by prioritizing

the timing of node behaviors and extracting underlying structural patterns from complex sequential evolution. To this end, we reformulate such temporal graph learning as a signal processing problem and propose SeqFilter, a simple yet robust neural network for link prediction. SeqFilter comprises two modules: a node rhythm memory and a frequency-selective structure encoder. For the former, we fundamentally shift the emphasis from *who-based* memorization to *when-based* rhythm recognition and design a learnable filter network that relies on absolute timestamps. This module generates historical node rhythm embeddings, which are integrated with recency-aware node state for node rhythm embedding representation. This can capture each node’s interaction rhythm over time (Fig. 1b.2), while preserving recency awareness. For the latter, we propose a *local-global-local* filtering paradigm to handle complex sequential structure evolution characterized by low repetition and potentially frequent updates. Upon it, we design a frequency-selective structure encoder that integrates local denoising, adaptive global adjustment, and local correlation learning. We prove that our structure encoder approximates the optimal linear denoiser, facilitating true underlying structural pattern extraction. Last, SeqFilter fuses the rhythm and structure embeddings to generate the node representation, eliminating redundant feature learning. SeqFilter theoretically functions as a composite filter; empirically, it learns structured filter weights (see Figs. 1b.1 and 1b.3).

We make the following contributions in this work:

- We propose a simple yet robust neural network to model complex sequential dynamics. Theoretically, we prove that SeqFilter functions as a composite filter and approximates the optimal linear denoiser, enabling it to effectively extract underlying temporal and structural patterns from complex sequential dynamics.
- By focusing on *when* nodes interact and *how* underlying structure evolves, we propose a node rhythm memory and a frequency-selective structure encoder for sequential dynamic modeling. They set SeqFilter apart from existing T-GNNs, eliminating feature redundancy, avoiding any assumptions, and preserving robustness to abrupt/outlier events.

¹We apply a 128-point Fast Fourier Transform (FFT) to the GRU’s hidden-to-hidden weight matrix from the best TGN checkpoint and visualize the positive-frequency magnitudes as a seaborn heatmap.

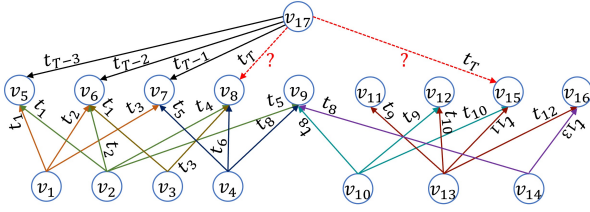


Figure 2: A toy example of sequential dynamics [54].

- Extensive experiments on 8 datasets show that SeqFilter outperforms 11 baselines, yielding an average MRR gain of 15.82% while incurring minimal overhead in both memory usage and runtime. Frequency-domain visualization of its learned weights across temporal graphs illustrates that the node rhythm memory captures a *uniform* behavior rhythm, whereas the structure encoder adapts *unique* structural evolution, highlighting the effectiveness of the proposed SeqFilter.

2 Retrospective

2.1 Preliminaries

DEFINITION 1 (TEMPORAL GRAPH). A temporal graph $G = (V, E)$ is constructed by a sequence of temporal events, where V is the node set and E is the edge set. $G = \{\delta(t_1), \delta(t_2), \dots\}$, which is ordered by timestamps. Each temporal interaction event is a tuple $\delta(t) = (v_i, v_j, e_{ij}(t), t)$ representing an edge, where v_i and v_j are node features of v_i and v_j (if available), $e_{ij}(t)$ denotes edge features (if available), and t is the timestamp at which $\delta(t)$ happens.

Following the TGB-Seq benchmark [54], we focus on temporal graphs with complex sequential dynamics commonly found in real dynamic systems. We provide a toy example of temporal graphs with sequential dynamics in Fig. 2.

Problem Statement. Our aim is to capture complex sequential dynamics by modeling how nodes' intentions evolve from their historical interactions to predict *unseen* future interactions. For example, given the recent events involving node v_{17} in Fig. 2, including $(v_{17}, v_7, e_{17,7}(t_{T-1}), t_{T-1})$, we predict whether the unseen interactions (e.g., $(v_{17}, v_8, e_{17,8}(t_T), t_T)$) will occur, evaluating the prediction against one or multiple negatives.

2.2 Rethinking Mainstream T-GNNs for Sequential Dynamics

Memory module in existing T-GNNs. To capture temporal dependencies, the memory module is designed to store node states prior to structure learning, specifically tailored for temporal graph learning. Existing designs aim to summarize the exact neighbor information of each node into a fixed-size vector, referred to as *memory*. Upon the arrival of a new temporal event $\delta(t) = (v_i, v_j, e_{ij}(t), t)$, the memories of nodes v_i and v_j are updated accordingly. For instance, the memory update for node v_i is defined as:

$$\mathbf{m}_i(t) \leftarrow \text{MEM}(\mathbf{m}_i(t'), \mathbf{v}_i, \mathbf{m}_j(t'), \mathbf{e}_{ij}(t), \Delta t), \quad (1)$$

where $\text{MEM}(\cdot)$ denotes the update function, typically implemented by recurrent neural networks such as a Gated Recurrent Unit (GRU) [18,

27]. In Eq. (1), $\mathbf{e}_{ij}(t)$ is the edge feature, $\mathbf{m}_*(t')$ is the previous memories, Δt is relative time interval, and $\mathbf{v}_i$ is the node feature.

Rethinking-1: What should be memorized? According to Eq. (1), existing T-GNNs predominantly rely on memorizing exact neighbor information for each node, such as neighbor memories $\mathbf{m}_j(t')$ and edge features $\mathbf{e}_{ij}(t)$, implicitly assuming periodic interactions with fixed neighbors, which seldom holds. The unstructured weight responses in Fig. 1a.2 empirically confirm it as sequential dynamic graphs rarely repeat edges (six of eight datasets show 0% recurrence). Furthermore, existing methods overlook temporal rhythms, yet these patterns are more telling than the exact neighbor information. On YouTube, for example, users follow regular cycles, such as watching videos every evening, but rarely click the same video each time. Effective models should therefore prioritize when interactions occur rather than memorizing whom nodes interact with.

Structure encoder/Co-neighbor encodings in existing T-GNNs.

Given a newly-arrived temporal edges $\delta(t) = (v_i, v_j, \mathbf{e}_{ij}(t), t)$, structure encoder is to sample and aggregate k historical neighbors of node v_i and v_j before time t , generating their structure embeddings. The aggregation for node v_i could be formulated as:

$$\mathbf{h}_i(t) \leftarrow \text{AGG}(\mathbf{v}_i, \{\mathbf{v}_j, \mathbf{e}_{ij}(t), \Delta t, \mathbf{c}_{ij}(t)\} \mid v_j \in \mathcal{N}_i^k(t)), \quad (2)$$

where $\mathcal{N}_i^k(t)$ denotes neighbors of v_i before t and $\text{AGG}(\cdot)$ denotes an aggregation function. When node memory is available, $\mathbf{v}_i$ is replaced by a combination of memory embedding $\mathbf{m}_i(t)$ and node feature $\mathbf{v}_i$. The relative neighbor encoding $\mathbf{c}_{ij}(t)$ is optional; recent studies [6, 23, 34, 43, 57] leverage this encoding by computing correlations between the neighbors of nodes v_i and v_j to capture their structural relationships. Formally, we define it as:

$$\mathbf{c}_{ij}(t) \leftarrow \text{CO-NEG}(\mathcal{N}_i^k(t), \mathcal{N}_j^k(t)), \quad (3)$$

where $\text{CO-NEG}(\cdot)$ is the neighbor co-occurrence encoding function, e.g., computing the number of common neighbors [57]. TPNet [23] unifies different constructions of Eq. (3) into a format.

Rethinking-2: Is neighbor co-occurrence still effective for sequential dynamics? Recent studies have explored neighbor co-occurrence encoding techniques to improve link prediction performance, which rely on the assumption that nodes sharing more historical neighbors are more likely to interact, an effective assumption for temporal graphs with repeated edges. However, in graphs characterized by sequential dynamics, future interactions are primarily driven by previous sequential dependency. When the graph evolves sequentially and rapidly, neighbor co-occurrences of node pairs tend to become sparse over time, especially under general temporal sampling. For example, we analyze the Patent dataset using the DyGFormer [57], computing the average neighbor co-occurrence ratio within chronologically split batches (sampling all one-hop neighbors per node). The results show that the co-occurrence ratio of node pairs is below 1% across most batches. This sparsity likely explains why DyGFormer, which relies heavily on neighbor co-occurrence encoding, achieves inferior performance, up to 12% lower in MRR, compared to simpler GraphMixer, as reported in the TGB-Seq benchmark [54]. Although expanding the neighbor receptive field partially alleviates sparsity, it introduces additional noise and significantly increases computational overhead.

Rethinking-3: How can T-GNNs handle inherently abrupt or outlier interactions? Real-world temporal graphs naturally contain abrupt or outlier events. A high update rate (for example, Yelp records over 10 million timestamps) may amplify the visibility and impact of such outliers. Yet existing T-GNNs, whether built on basic neighbor aggregation or co-occurrence encodings, lack mechanisms to handle insignificant abrupt events. They therefore overfit these irregular interactions, miss the underlying structural patterns, and ultimately compromise model generalization.

3 The Proposed SeqFilter

To effectively handle temporal graphs characterized by sequential dynamics, two key points are essential: (1) prioritizing the timing (*when*) of node behaviors rather than the identity (*who*) of interaction neighbors; and (2) extracting underlying structural patterns from evolving sequential structures. To this end, we reformulate sequential dynamic modeling as a signal processing problem, regarding node representation learning as a filtering process. Specifically, we propose SeqFilter, containing two modules: a node rhythm memory and a frequency-selective structure encoder, which collaboratively avoids feature redundancy and any assumptions (see Fig. 1b). The node rhythm memory shifts away from memorizing exact node interactions towards capturing temporal rhythms of node behaviors. It can encode periodic activities, together with recency awareness, generating node rhythm embeddings. For the frequency-selective structure encoder, we propose a *local-global-local* filtering paradigm designed to selectively amplify or suppress specific frequency components within the neighbor spectrum for local correlation learning. Last, the rhythm and structure embeddings are fused to generate high-quality node representations.

3.1 Node Rhythm Memory

Motivated by **Rethinking-1**, we shift the emphasis of memory modeling from *whom* a node interacts with to *when* node behavior occurs, as interactions are rarely repeated between the same node pairs. To achieve this, we propose a Node Rhythm Memory module that: (1) removes dependence on exact historical neighbor identities and (2) eliminates redundant encoding in both memory and structure modules. Instead, we model the *temporal rhythm* of a node's behavior exclusively using absolute interaction timestamps. Given a temporal event $\delta(t) = (v_i, v_j, e_{ij}(t), t)$, we update the node rhythm memories for nodes v_i and v_j . Taking node v_i as an example, we construct its input by concatenating its previous rhythm memory with the current absolute timestamp encodings and update its node rhythm memory via a depthwise convolution as follows:

$$r'_i(t) = \text{Linear}(\text{DepthwiseConv1D}([\phi(t) \oplus r'_i(t')])) \quad (4)$$

where $\phi(\cdot)$ denotes a time encoding function [46] and $\oplus$ is the concatenation operator. $r'_i(t')$ is previous historical node rhythm memory. $\text{DepthwiseConv1D}(\cdot)$ acts as a channel-wise and independent filter [8, 15] for each temporal dimension, enabling it to handle heterogeneous updates across channels and capture both slow and fast rhythm patterns. The following Linear layer aggregates these filtered channels into the updated memories (e.g., $r'_i(t)$). As Eq. (4) uses absolute timestamps, it aligns events on a unified time axis, allowing rhythm patterns to emerge, even when a node interacts with

different neighbors, and making synchronized behaviors easy to detect. Thus, the node rhythm memory captures long-term behavioral regularities without relying on repeated interactions.

While long-term node rhythms may capture stable preferences, their relevance easily fades with inactivity. To handle this, we introduce a recency-aware node state term that conditions historical behaviors on the elapsed time since the previous interaction, reducing the risk that stale patterns overwhelm current predictions. Given each node $v_i(t)$, we obtain its current node rhythm embedding for node representation by

$$r_i(t) = \text{LayerNorm}(\text{DepthwiseConv1D}([r'_i(t') \oplus \phi(t - t')])) \quad (5)$$

where $\text{LayerNorm}(\cdot)$ is the Layer Normalization, t is the current timestamp, and t' is the previous timestamp. Eq. (5) exposes recency explicitly so that stale patterns do not dominate after inactivity. It generates current node rhythm embeddings for node representations, which integrates long-term behavioral regularities with recency awareness.

3.2 Frequency-Selective Temporal Structure Encoder

In sequential dynamics, temporal graphs evolve sequentially with few repeated edges, making it difficult for existing structure encoders, regardless of their reliance on the co-occurrence assumption, to capture underlying structural patterns (as noted in **Rethinking-2** and **Rethinking-3**). Their key limitation lies in their inability to handle abrupt interactions across neighbors. FreeDyG [34] partly mitigates this by using a frequency-enhanced MLP-Mixer to downweight weak signals, but it lacks an explicit mechanism to process abrupt events and thus compromises generalization. To address the challenge of complex structural evolution, we propose a *local-global-local* filtering paradigm to capture the evolving patterns of underlying graph structures. Upon it, we propose the Frequency-Selective Temporal Structure (FSTS) encoder, which cascades three learnable filters to locally denoise (insignificant) abrupt and outlier events, globally reweight frequency bands to highlight informative components, and learn local structure correlations across neighbors. Given a temporal event $\delta(t) = (v_i, v_j, e_{ij}(t), t)$, we first sample k neighbors for nodes v_i and v_j before timestamp t , obtaining $\mathcal{N}_i^k(t)$ and $\mathcal{N}_j^k(t)$, respectively. For node v_i , the input to FSTS encoder is constructed by concatenating edge features, relative time intervals, and neighbors' features: $x_{ij}(t) = [e_{ij}(t) \oplus \phi(\Delta t) \oplus v_j]$, where $j \in \mathcal{N}_i^k(t)$. Stacking these inputs across a mini-batch constructs the matrix $X(t)$, which we convert to the frequency domain via the Discrete Fourier Transform (DFT) [3]: $X(f) = \text{DFT}(X(t))$. Since abrupt interactions correspond to high-frequency components, we leverage an energy-based masking mechanism [11] to filter out high-frequency information by

$$\mathcal{X}_{\text{low}}(f) = \mathbb{I}(|X(f)|^2 > \tau) \odot X(f), \quad (6)$$

where $|\cdot|^2$ measures the squared magnitude of frequency responses, $\mathbb{I}(\cdot)$ is the indicator function, and $\odot$ represents element-wise multiplication. τ is the energy threshold to discern between underlying signal frequencies and abrupt/outlier ones. A larger τ more effectively filters abrupt or outlier signals. Based on Eq. (6), we can obtain essential frequency components without significant noise.

In practice, sequential dynamics cover time ranges from minutes to months, so different nodes and datasets exhibit responses at different bands (daily, weekly, monthly, etc.). We aim to adaptively enhance informative frequency bands before local structure learning, where we apply a complex-valued linear projection over $X_{\text{low}}(f)$:

$$X_{\text{proj}}(t) = \text{IDFT}(X_{\text{low}}(f) \cdot \mathcal{W}_{\mathbb{C}}), \quad \mathcal{W}_{\mathbb{C}} = \mathbf{W}_r + i \cdot \mathbf{W}_i, \quad (7)$$

where $\text{IDFT}(\cdot)$ is the inverse DFT. $\mathbf{W}_r$ and $\mathbf{W}_i$ are the real and imaginary parts of the complex-valued weight matrix $\mathcal{W}_{\mathbb{C}}$. This projection enables frequency-aware representation learning by adaptively adjusting the model's response to different frequency bands. After the local-global filtering process, it helps mitigate overfitting to abrupt events while preserving diverse periodic patterns essential for subsequent underlying structure learning. We then capture local structural correlations by applying a standard convolution over $X_{\text{proj}}(t)$, which is formulated as:

$$X_{\text{out}}(t) = \text{Conv1D}(X_{\text{proj}}(t)), \quad (8)$$

$$Z(t) = \tanh(\mathbf{R}(t) + \text{MeanPooling}(X_{\text{out}}(t))), \quad (9)$$

where $\mathbf{R}(t)$ is current rhythm embeddings for batch nodes, $\tanh(\cdot)$ is the activation function, and $\text{MeanPooling}(\cdot)$ denotes the mean pooling. $\text{Conv1D}(\cdot)$ is the convolution layer, enabling the model to capture local structural correlations. Based on Eq. (9), we obtain the node embeddings (e.g., $z_i(t)$) by fusing node rhythm embeddings $\mathbf{R}(t)$ and node structure embeddings $X_{\text{out}}(t)$. In summary, the FSTS encoder integrates three key components: local denoising in the frequency domain, adaptive global frequency response adjustment in the frequency domain, and local correlation learning in the time domain. Together, they function as a cascade of learnable filters, which is formally introduced by the following theorem.

THEOREM 1 (CASCADE FILTERING OF FSTS ENCODER). *Let $X(t)$ be the stacked neighbor representation for a mini-batch at time t , and define its spectrum by the discrete Fourier transform $X(f) = \text{DFT}(X(t))$. The output of the Frequency-Selective Temporal Structure (FSTS) encoder in the frequency domain is*

$$X_{\text{out}}(f) = \mathcal{H}_{\text{FSTS}}(f) \cdot X(f) = \mathcal{H}_{\mathbb{C}}(f) \cdot \mathcal{H}_{\mathbb{P}}(f) \cdot \mathcal{H}_{\mathbb{L}}(f) \cdot X(f), \quad (10)$$

where $\mathcal{H}_{\text{FSTS}}(f)$ denotes the overall frequency response; $\mathcal{H}_{\mathbb{C}}(f)$, $\mathcal{H}_{\mathbb{P}}(f)$, and $\mathcal{H}_{\mathbb{L}}(f)$ denote the frequency response of a convolution FIR filter in Eq. (8), a complex-valued linear projection that globally re-weights frequency bands in Eq. (7), and a low-pass filter induced by high-frequency denoising in Eq. (6), respectively.

The proof of Theorem 1 is provided in the Appendix A.1. As demonstrated in Theorem 1, the FSTS encoder functions as a cascade of filters for temporal structure learning: $\mathcal{H}_{\mathbb{L}}(f)$ serves as a low-pass filter that suppresses high-frequency abrupt and outlier interactions while preserving dominant components in the neighbor spectrum (*local*); $\mathcal{H}_{\mathbb{P}}(f)$ adaptively assigns data-driven gains to individual frequency bands (*global*); and, the time-domain convolution acts as a FIR filter that captures local correlations and smooths short-term fluctuations (*local*). This naturally raises an important question: How does our structure encoder capture underlying structural patterns from sequential evolution for modelling temporal structure? We answer it by presenting the following theorem.

THEOREM 2 (ε -OPTIMAL LINEAR DENOISING BY FSTS ENCODER). *Let $S(t)$ and $N(t)$ be zero-mean, wide-sense stationary processes*

Table 1: Statistics of datasets with sequential dynamics.

Dataset	# Nodes (U/I)	# Edges	# Timestamps	Repeat ratio
ML-20M	100,785 / 9,646	14,494,325	9,993,250	0
Taobao	760,617 / 863,016	18,853,792	139,171	0.17
Yelp	1,338,688 / 405,081	19,760,293	14,646,734	0.25
GoogleLocal	206,244 / 267,336	1,913,967	1,771,060	0
Flickr	233,836	7,223,559	134	0
YouTube	402,422	3,288,028	203	0
Patent	2,241,784	12,749,824	1,632	0
WikiLink	1,361,972	34,163,774	2,198	0

with power spectral densities $S_s(f)$ and $S_n(f)$, respectively. We define $X(t) = (h * S)(t) + N(t)$, and $P(f) = |H(f)|^2 \cdot S_s(f) + S_n(f)$, with known h and spectrum $H(f)$. $W_{\text{opt}}(f) = \frac{H^*(f) \cdot S_s(f)}{P(f)}$ minimizes the mean-square error $\text{MSE}_{\min} = \int_0^1 \frac{S_s(f) \cdot S_n(f)}{P(f)} df$, where $H^*(f)$ is the conjugate of $H(f)$. For any $\varepsilon > 0$, the FSTS encoder can choose its parameters $\mathcal{H}_{\text{FSTS}}(f)$ such that

$$\sup_{f \in [0,1]} |\mathcal{H}_{\text{FSTS}}(f) - W_{\text{opt}}(f)| < \varepsilon, \quad (11)$$

which implies $\text{MSE}_{\text{FSTS}} - \text{MSE}_{\min} = \int_0^1 |\mathcal{H}_{\text{FSTS}}(f) - W_{\text{opt}}(f)|^2 \cdot P(f) df < \varepsilon^2 \int_0^1 P(f) df$.

The proof of Theorem 2 is provided in the Appendix A.2. Theorem 2 asserts that the FSTS encoder achieves a mean-squared error within $\varepsilon^2 \int_0^1 P(f) df$ of the Wiener optimal solution, converging to the optimum as $\varepsilon \rightarrow 0$. FSTS encoder keeps the Wiener filter's strength: preserving true structural patterns that drive sequential evolution by reliably suppressing noise and abrupt events. Unlike the fixed Wiener filter, the FSTS encoder is learnable: it does not assume that signal and noise are uncorrelated and processes feature channels jointly. Thus, the FSTS encoder offers a near-optimal, learnable approximation for uncovering underlying structures in complex sequential dynamics, where convolutions or existing structure encoders often fail due to abrupt events or irregular interactions.

4 Experiments

4.1 Experimental Setting

Datasets. We collect eight temporal graphs with complex sequential dynamics from the latest TGB-Seq benchmark [54]. These comprise four bipartite graphs (ML-20M, Taobao, Yelp, and GoogleLocal) and four non-bipartite graphs (Flickr, YouTube, Patent, and WikiLink). Table 1 reports their key statistics, such as graph update frequency (timestamps) and edge repeat ratio.

Baselines and evaluation metrics. We compare the proposed SeqFilter against 11 state-of-the-art T-GNNs, covering four groups: T-GNNs with memory (JODIE [18], DyRep [37], and TGN [27]), T-GNNs with neighbor co-occurrence (CAWN [43], DyGFormer [57], FreeDyG [34], CNEN [6], and TPNet [23]), simple T-GNNs (TGAT [46], TCL [40], and GraphMixer [9]), and frequency-enhanced T-GNNs (FreeDyG [34]). More baseline details are provided in Appendix B.2. We evaluate *future link prediction* using 100 negatives per positive and report Mean Reciprocal Rank (MRR) in the ranking setting; in the binary setting, we report the Area Under the ROC Curve

Table 2: MRR (%) of twelve T-GNNs on eight TGB-Seq datasets. The first and second place rankings are highlighted accordingly. Results marked with an asterisk are collected from [54].

Datasets	ML-20M	Taobao	Yelp	GoogleLocal	Flickr	YouTube	Patent	WikiLink	Avg
JODIE* [18]	21.16 $\pm$ 0.73	48.36 $\pm$ 2.18	<u>69.88$\pm$0.31</u>	41.86 $\pm$ 1.49	46.21 $\pm$ 0.83	41.67 $\pm$ 2.86	24.60 $\pm$ 0.38	57.94 $\pm$ 1.33	43.96
DyRep* [37]	19.00 $\pm$ 1.69	40.03 $\pm$ 2.40	57.69 $\pm$ 1.05	37.73 $\pm$ 1.34	38.04 $\pm$ 4.19	35.12 $\pm$ 4.13	21.01 $\pm$ 1.14	42.63 $\pm$ 1.33	36.41
TGAT* [46]	10.47 $\pm$ 0.20	TLE	TLE	19.78 $\pm$ 0.24	23.53 $\pm$ 3.35	43.56 $\pm$ 2.53	8.49 $\pm$ 0.18	TLE	21.17
TGN* [27]	23.99 $\pm$ 0.20	60.28 $\pm$ 0.54	69.79 $\pm$ 0.24	54.13$\pm$1.97	46.03 $\pm$ 6.78	55.16 $\pm$ 5.89	22.83 $\pm$ 2.25	62.94 $\pm$ 2.16	<u>53.14</u>
CAWN* [43]	12.31 $\pm$ 0.02	TLE	25.71 $\pm$ 0.09	18.26 $\pm$ 0.02	48.69 $\pm$ 6.08	47.55 $\pm$ 1.08	12.34 $\pm$ 0.47	TLE	26.34
TCL* [40]	12.04 $\pm$ 0.02	31.55 $\pm$ 0.03	24.39 $\pm$ 0.67	18.30 $\pm$ 0.02	40.00 $\pm$ 1.76	50.17 $\pm$ 1.98	10.60 $\pm$ 1.75	43.02 $\pm$ 2.16	28.76
GraphMixer* [9]	21.97 $\pm$ 0.17	31.54 $\pm$ 0.02	33.96 $\pm$ 0.19	21.31 $\pm$ 0.14	45.01 $\pm$ 0.08	58.87 $\pm$ 0.12	18.97 $\pm$ 2.54	48.57 $\pm$ 0.02	32.49
DyGFormer* [57]	TLE	TLE	21.68 $\pm$ 0.20	18.39 $\pm$ 0.02	49.58 $\pm$ 2.87	46.08 $\pm$ 3.44	14.20 $\pm$ 2.93	TLE	29.99
CNEN [6]	18.16 $\pm$ 0.32	48.76 $\pm$ 0.01	56.13 $\pm$ 0.02	<u>49.27$\pm$0.25</u>	55.94 $\pm$ 0.37	45.49 $\pm$ 0.32	17.76 $\pm$ 6.76	64.80 $\pm$ 1.02	41.12
TPNet [23]	24.12 $\pm$ 0.23	<u>67.03$\pm$0.33</u>	41.07 $\pm$ 0.02	35.20 $\pm$ 0.21	<u>72.91$\pm$0.03</u>	<u>72.67$\pm$0.12</u>	<u>29.72$\pm$0.61</u>	<u>65.78$\pm$0.34</u>	51.06
FreeDyG [34]	<u>24.17$\pm$0.12</u>	43.40 $\pm$ 0.02	38.45 $\pm$ 0.08	20.01 $\pm$ 0.27	61.63 $\pm$ 0.16	59.70 $\pm$ 0.22	5.97 $\pm$ 0.21	TLE	36.19
SeqFilter	64.46$\pm$0.60	83.44$\pm$0.11	89.61$\pm$0.06	47.52 $\pm$ 0.01	89.02$\pm$0.52	87.15$\pm$0.28	45.68$\pm$0.13	75.99$\pm$0.51	72.85

(AUC) [33, 50, 54]. Any model exceeding 24 hours per epoch or 48 hours overall is flagged TLE (Time Limit Exceeded). Results are the mean and standard deviation over three runs. Detailed training configurations are available in Appendix B.3.

4.2 Effectiveness Evaluation

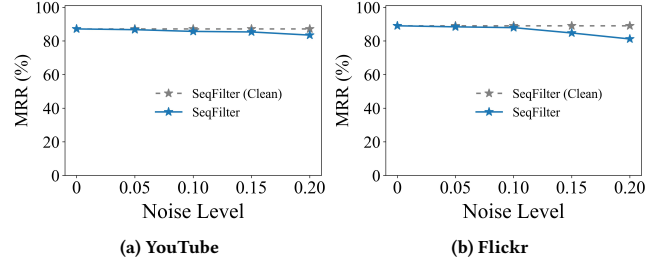
We evaluate the performance of SeqFilter compared to eleven baselines using future link prediction under the MRR metric on eight temporal graphs with sequential dynamics.

Obs-1: Overall effectiveness and generalization. It is observed from Table 2 that SeqFilter achieves an average MRR improvement of 15.82% over 11 baselines, demonstrating that a filter-driven network effectively captures temporal and structural patterns in sequential dynamics, resulting in stronger generalization to unseen scenarios. Memory-based models (e.g., JODIE, TGN) and neighbor co-occurrence models (e.g., DyGFormer, TPNet) perform well only in isolated scenarios, with none showing consistent strength. SeqFilter outperforms the best memory models by up to 40.47%, highlighting that encoding when a node interacts is more effective than memorizing whom a node interacts with. Neighbor co-occurrence models do not perform well on long-term or sparse temporal graphs, where shared neighbors are rare. SeqFilter outperforms FreeDyG, whose frequency filtering lacks explicit abrupt event suppression, by up to 51.16% in MRR. Compared to simpler T-GNNs like GraphMixer, SeqFilter demonstrates superior generalization and effectiveness by learning filter weights that preserve underlying temporal and structural patterns, a capability that existing baselines lack.

We report results for future link prediction in the binary setting in Table 4 in Appendix B.4, and present results on two general temporal graphs rich in repeated edges in Table 5 in Appendix B.5.

4.3 Robustness Evaluation

To assess the robustness of our SeqFilter, we conduct experiments on synthetic noisy temporal graphs by injecting varying levels of noise into the clean training data. We report future link prediction results in terms of MRR on the YouTube and Flickr datasets, where noise level ranges from 0 to 0.2.

**Figure 3: Robustness evaluation with different-level noise regarding MRR on two datasets.**

Obs-2: Robustness to noisy scenarios. As shown in Fig. 3, SeqFilter maintains relatively stable performance under increasing noise levels. Concretely, SeqFilter suffers less than a 5% drop even with 15% injected noise, thereby empirically substantiating the denoising guarantee stated in Theorem 2. When the noise level rises to 20% (657,605 outlier edges on YouTube), SeqFilter still delivers competitive results (less than 10% MRR drop), confirming its ability to capture underlying structural patterns of complex evolutions for real-world temporal graphs.

4.4 Efficiency Evaluation

We evaluate the efficiency of SeqFilter against 11 baselines using two metrics across two datasets (*i.e.*, Patent and Yelp). All methods are run with the same batch size in our environment.

Obs-3: Time efficiency. As shown in Fig. 4, SeqFilter achieves the fastest per-epoch training time among 12 T-GNNs on two datasets due to a lightweight network. SeqFilter is up to 26 $\times$ faster than TGN, whose tightly coupled memory and structure modules slow down training. This speedup is largely driven by SeqFilter’s architecture and efficient feature utilization. SeqFilter is also an *order of magnitude faster* than FreeDyG, highlighting the efficiency of its purely filter-driven design. Compared with efficient T-GNNs

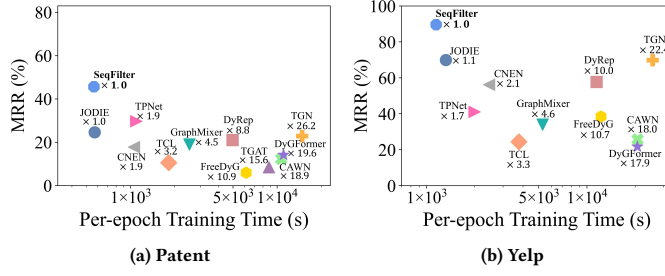


Figure 4: Runtime comparison on two datasets.

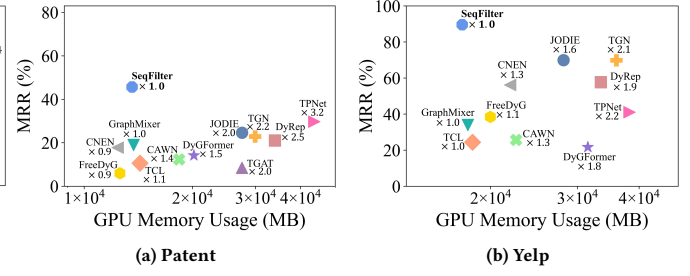


Figure 5: GPU memory usage comparison on two datasets.

Table 3: Ablation study in terms of MRR (%) on four datasets.

Dataset	SeqFilter	w/o Low-pass	w/o Global	w/o Conv1D	w/o Rhythm	w/o Recency	w RNN
ML-20M	64.46	21.08	60.69	7.88	57.45	61.75	63.08
Taobao	83.44	31.14	64.30	31.39	80.23	82.44	81.39
Flickr	89.02	45.41	86.91	70.32	82.26	83.12	85.37
YouTube	87.15	59.88	83.91	65.55	81.76	83.88	84.76

such as JODIE, TPNet, and CNEN, SeqFilter delivers comparable or slightly faster runtime.

Obs-4: Memory efficiency. Fig. 5 shows that SeqFilter consumes lower GPU memory during training among 12 T-GNNs, where it takes up to 221% less memory than the most memory-intensive baseline (TPNet) on the Patent dataset. This memory efficiency comes from avoiding costly neighbor co-occurrence caches and leveraging a decoupled design that reduces parameter overhead.

4.5 Ablation Study

We investigate the effect of individual design on four datasets. We conduct 6 variants: (1) removing high-frequency denoising (*w/o Low-pass*); (2) removing complex-valued linear projection (*w/o Global*); (3) removing standard Conv1D in structure encoder (*w/o Conv1D*); (4) removing long-term rhythm (*w/o Rhythm*); (5) removing recency awareness (*w/o Recency*); (6) replacing depthwise convolution with RNN in rhythm memory (*w RNN*). We report results in Table 3.

Obs-5: Effectiveness of frequency-selective structure encoder. The proposed FSTS encoder, designed as a cascade of learnable filters, proves essential for extracting underlying structural patterns. Each component contributes independently, with performance gains of 41.64%, 7.07%, and 37.23% attributed to local denoising, adaptive global adjustment, and local correlation learning, respectively. These indicate the effectiveness of a composite filter for temporal structure learning.

Obs-6: Effectiveness of rhythm memory. Rhythm memory plays a critical role in sequential dynamic modeling by capturing the periodic behavior rhythms of each node along with recency awareness. Removing long-term rhythms and recency modeling leads to 5.59% and 3.22% performance drop, respectively. Moreover, rhythm memory update in SeqFilter outperforms RNN-based updates, achieving an average improvement of 2.37%.

4.6 Visualization

We visualize the mean convolution weights of the node rhythm memory and the frequency-selective structure encoder in the frequency domain on two datasets. Specifically, we load the convolution weights, take a 128-point FFT to obtain single-sided magnitudes, stack them into a 2D array, and heatmap the kernels' frequency emphasis. As shown in Figs. 6 and 1b, the two modules emphasize different frequency bands, revealing distinct temporal and structural patterns.

Obs-7: Uniform periodic patterns in node rhythms. In Figs. 6a and 1b.1, the weights of rhythm memory consistently focus on mid-frequency bands across Taobao and YouTube, revealing periodic temporal patterns in node behaviors (e.g., cycles with periods of about 4 to 7 days on YouTube, considering its timestamp resolution). The corresponding heatmaps (e.g., Fig. 6b) show kernel-level variations, capturing the diverse temporal behaviors across nodes.

Obs-8: Unique structural patterns across temporal graphs. The frequency responses of weights in structure encoders vary by two datasets: on Taobao, it preserves low-frequency signals that capture long-term structural evolution, whereas on YouTube, it amplifies mid- and high-frequency bands to follow periodic structural patterns and meaningful, abrupt structural changes (Figs. 6c and 1b.3 (right)). These confirm that SeqFilter's filter-driven design adapts to structural changes, preserving underlying structural patterns. Looking at the heatmap of our structure encoder of the Taobao dataset (Fig. 6d), the rightmost columns, representing high-frequency components, remain relatively dark, revealing that the structure encoder intentionally suppresses high-frequency signals (insignificant abrupt or outlier events), empirically supporting our motivation for structure learning in complex sequential dynamics.

4.7 Parameter Sensitivity Analysis

To assess the hyperparameter sensitivity of SeqFilter, we vary the energy threshold τ and the number of neighbors on two datasets.

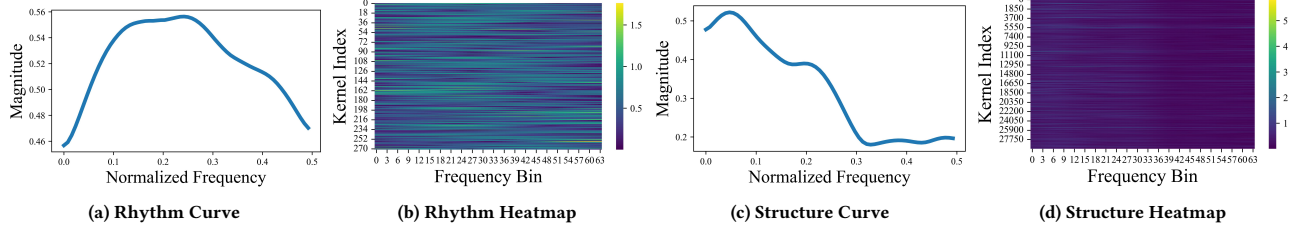


Figure 6: The visualization of rhythm and structure convolution kernels in the frequency domain on the Taobao dataset.

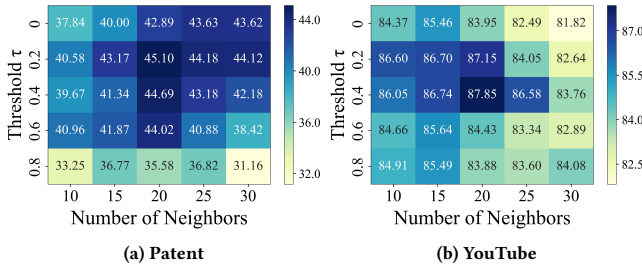


Figure 7: The heatmap values in MRR (%) of SeqFilter in different parameter settings on two datasets.

Obs-9: Stable performance within a parameter range. In Fig. 7, we report the MRR results on Patent and YouTube datasets towards these two hyperparameters. It is observed that an insufficiently large τ admits high-frequency components that obscure underlying structural patterns, while an excessively large τ suppresses informative mid- and high-frequency contents, both resulting in a performance drop. SeqFilter generally performs well across datasets when $\tau \in [0.2, 0.4]$. Additionally, using fewer neighbors diminishes structural context and degrades performance; performance plateaus once the neighbor count exceeds 20. Furthermore, a larger neighbor count requires a correspondingly larger τ , suggesting that more neighbors may introduce more noise. We provide hyperparameter recommendations in Appendix B.3.

5 Related Work

Temporal graph neural networks. T-GNNs have become a powerful framework for link prediction, offering strong capabilities in modeling temporal and structural evolutions [6, 7, 13, 16, 21, 25, 28, 29, 31, 32, 34, 39, 44, 48–51, 57–60, 62]. Early methods [18, 27, 37, 42] introduced node memory modules to store each node’s historical interactions and then incorporated those memory vectors into subsequent structure learning. While retaining exact neighbor information can improve predictions for repeated edges, it often leads to poor generalization in unseen scenarios, especially when structure encoders repeatedly learn the same neighbor features. Recent methods explored neighbor co-occurrence encoding schemes [6, 23, 34, 43, 57] to capture relative relationships of nodes from historical neighbors under the assumption that nodes sharing more neighbors are more likely to interact in the future. These

schemes boost performance on graphs with high edge repeats, but they suffer performance drops on temporal graphs driven by sequential dynamics. Some efforts with simple neural architectures (e.g., MLP Mixer) [9] have achieved state-of-the-art results on general benchmarks. However, existing T-GNNs face challenges in modeling complex sequential dynamics, a recently emerging issue in temporal graph learning [54], where interactions are driven by sequential dependency and the goal is to predict unseen edges.

Frequency-enhanced learning. Recent work has explored the use of frequency-domain networks to enhance learning across various domains [1, 20, 22, 34, 41, 47, 52, 53, 55], including temporal graph learning, time series forecasting, and computer vision. For example, FreeDyG [34] incorporated a frequency-enhanced MLP-Mixer to adaptively downweight insignificant signals while employing neighbor encodings for node representations on temporal graphs. L-STEP [35] applied a frequency-enhanced linear transformation to positional encodings to store approximate position information for temporal link prediction. However, even on the smallest TGB-Seq dataset, L-STEP runs out of memory on an NVIDIA A100 (40 GB). Different from them, our method is to design a filter-driven network that directly learns filter weights to capture temporal and structural patterns inherent in sequential dynamics.

6 Conclusion

In this paper, we address the challenge of modeling complex sequential dynamics on temporal graphs by drawing inspiration from signal processing. We propose a simple yet robust neural network that learns composite filter weights to generate node representations. To capture temporal dependencies, we shift the modeling focus from *whom* a node interacts with to *when* it interacts, designing a node rhythm memory that effectively encodes periodic behavior rhythms—an effect we find to be remarkably consistent across domains. Furthermore, we propose a frequency-selective structure encoder based on a *local-global-local* filtering paradigm, which captures underlying structural patterns from complex sequential evolution for effective structure learning. Theoretically, we prove that our structure encoder functions as a cascade of learnable filters, approximating the optimal linear denoiser. The final node representation is obtained by fusing the outputs of the rhythm memory with recency awareness and the structure encoder. Extensive experiments on eight temporal graphs with sequential dynamics demonstrate the effectiveness, efficiency, and robustness of our SeqFilter. Visualization of the learned weights validates the soundness of the proposed model design.

7 Acknowledgments

The work of Ying Zhang was supported by the Zhejiang Federation of Humanities and Social Sciences under Grant 25SYS06ZD. The work of Wenjie Zhang was supported by the Australian Research Council (ARC) under Grant DP230101445 and Grant FT210100303, and Australian Research Council Centre of Excellence for Mathematical Modelling of Cellular Systems under Grant CE230100001. The work of Xuemin Lin was supported by the National Natural Science Foundation of China (NSFC) under Grant U2241211. The work of Lei Chen is partially supported by the National Key Research and Development Program of China under Grant No. 2023YFF0725100, National Science Foundation of China (NSFC) under Grant No. U22B2060, Guangdong-Hong Kong Technology Innovation Joint Funding Scheme Project No. 2024A0505040012, AOE Project AoE/E-603/18, Theme-based project TRS T41-603/20R, CRF Project C2004-21G, Key Areas Special Project of Guangdong Provincial Universities 2024ZDZX1006, Guangdong Province Science and Technology Plan Project 2023A0505030011, Guangzhou municipality big data intelligence key lab, 2023A03J0012, HKUST(GZ)-CMCC(Guangzhou Branch) Metaverse Joint Innovation Lab under Grant No. P00659, Hong Kong ITC ITF grant PRP/004/22FX, Zhujiang scholar program 2021JC02X170, HKUST-Webank joint research lab.

References

- [1] Anson Bastos, Kuldeep Singh, Abhishek Nadgeri, Manish Singh, and Toyotaro Suzumura. 2024. Beyond Spatio-Temporal Representations: Evolving Fourier Transform for Temporal Graphs. In *International Conference on Learning Representations*.
- [2] Paolo Boldi, Marco Rosa, Massimo Santini, and Sebastiano Vigna. 2011. Layered label propagation: A multiresolution coordinate-free ordering for compressing social networks. In *The ACM Web Conference*. 587–596.
- [3] Georg Bruun. 1978. z-transform DFT filters and FFT's. *IEEE Transactions on Acoustics, Speech, and Signal Processing* 26, 1 (1978), 56–63.
- [4] Lei Cai, Zhengzhang Chen, Chen Luo, Jiaping Gui, Jingchao Ni, Ding Li, and Haifeng Chen. 2021. Structural temporal graph neural networks for anomaly detection in dynamic graphs. In *ACM International Conference on Information and Knowledge Management*. 3747–3756.
- [5] Meeyoung Cha, Alan Mislove, and Krishna P Gummadi. 2009. A measurement-driven analysis of information propagation in the flickr social network. In *The ACM Web Conference*. 721–730.
- [6] Ke Cheng, Peng Linzhi, Junchen Ye, Leilei Sun, and Bowen Du. 2024. Co-Neighbor Encoding Schema: A Light-cost Structure Encoding Method for Dynamic Link Prediction. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 421–432.
- [7] Ke Cheng, Linzhi Peng, Pengyang Wang, Heng Chang, Junchen Ye, and Bowen Du. 2025. On the Scalability of Temporal Relative Positional Encoding for Dynamic Link Prediction. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 298–309.
- [8] François Chollet. 2017. Xception: Deep learning with depthwise separable convolutions. In *IEEE Conference on Computer Vision and Pattern Recognition*. 1251–1258.
- [9] Weilin Cong, Si Zhang, Jian Kang, Baichuan Yuan, Hao Wu, Xin Zhou, Hanghang Tong, and Mehrdad Mahdavi. 2023. Do We Really Need Complicated Model Architectures For Temporal Networks?. In *International Conference on Learning Representations*.
- [10] Nur Nasuha Daud, Siti Hafizah Ab Hamid, Muntadher Saadoun, Firdaus Sahran, and Nor Badrul Anuar. 2020. Applications of link prediction in social networks: A review. *Journal of Network and Computer Applications* 166 (2020), 102716.
- [11] Emadeldien Eldele, Mohamed Ragab, Zhenghua Chen, Min Wu, and Xiaoli Li. 2024. TSLANet: Rethinking Transformers for Time Series Representation Learning. In *International Conference on Machine Learning*. PMLR, 12409–12428.
- [12] Alessio Gravina and Davide Bacciu. 2024. Deep learning for dynamic graphs: models and benchmarks. *IEEE Transactions on Neural Networks and Learning Systems* (2024).
- [13] Alessio Gravina, Giulio Lovisotto, Claudio Gallicchio, Davide Bacciu, and Claas Grohnfeldt. 2024. Long range propagation on continuous-time dynamic graphs. In *International Conference on Machine Learning*. 16206–16225.
- [14] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. 2017. Neural collaborative filtering. In *The ACM Web Conference*. 173–182.
- [15] Andrew G Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. 2017. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv* (2017).
- [16] Yunming Hui, Inez Maria Zwetsloot, Simon Trimbom, and Stevan Rudinac. 2025. Domain-Informed Negative Sampling Strategies for Dynamic Graph Embedding in Meme Stock-Related Social Networks. In *The ACM on Web Conference*. 518–529.
- [17] Seyed Mehran Kazemi, Rishab Goel, Kshitij Jain, Ivan Kobyzev, Akshay Sethi, Peter Forsyth, and Pascal Poupart. 2020. Representation Learning for Dynamic Graphs: A Survey. *Journal of Machine Learning Research* 21 (2020), 70:1–70:73.
- [18] Srijan Kumar, Xikun Zhang, and Jure Leskovec. 2019. Predicting Dynamic Embedding Trajectory in Temporal Interaction Networks. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. ACM, 1269–1278.
- [19] Jiacheng Li, Jingbo Shang, and Julian McAuley. 2022. UCTopic: Unsupervised Contrastive Learning for Phrase Representations and Topic Mining. In *Annual Meeting of the Association for Computational Linguistics*. 6159–6169.
- [20] Shuhao Li, Yue Cui, Jingyi Xu, Libin Li, Linghai Meng, Weidong Yang, Fan Zhang, and Xiaofang Zhou. 2025. Unifying lane-level traffic prediction from a graph structural perspective: Benchmark and baseline. *IEEE Transactions on Knowledge and Data Engineering* (2025).
- [21] Shuhao Li, Weidong Yang, Yue Cui, Xiaoxing Liu, Linghai Meng, Lipeng Ma, and Fan Zhang. 2025. Fine-Grained Traffic Inference from Road to Lane via Spatio-Temporal Graph Node Generation. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. 1529–1540.
- [22] Yingxuan Li, Yuanyuan Xu, Xuemin Lin, Wenjie Zhang, and Ying Zhang. 2025. Ranking on Dynamic Graphs: An Effective and Robust Band-Pass Disentangled Approach. In *The ACM Web Conference*. 3918–3929.
- [23] Xiaodong Lu, Leilei Sun, Tongyu Zhu, and Weifeng Lv. 2024. Improving Temporal Link Prediction via Temporal Walk Matrix Projection. In *Advances in Neural Information Processing Systems*, Vol. 37. 141153–141182.
- [24] Alan Mislove, Massimiliano Marcon, Krishna P Gummadi, Peter Druschel, and Bobby Bhattacharjee. 2007. Measurement and analysis of online social networks. In *ACM SIGCOMM Conference on Internet Measurement*. 29–42.
- [25] Zhiqiang Pan, Chen Gao, Fei Cai, Wanyu Chen, Xin Zhang, Honghui Chen, and Yong Li. 2025. On the Cross-Graph Transferability of Dynamic Link Prediction. In *The ACM on Web Conference*. 4101–4110.
- [26] Meng Qin and Dit-Yan Yeung. 2023. Temporal link prediction: A unified framework, taxonomy, and review. *Comput. Surveys* 56, 4 (2023), 1–40.
- [27] Emanuele Rossi, Ben Chamberlain, Fabrizio Frasca, Davide Eynard, Federico Monti, and Michael Bronstein. 2020. Temporal Graph Networks for Deep Learning on Dynamic Graphs. In *ICML Workshop on Graph Representation Learning*.
- [28] Aravind Sankar, Yanhong Wu, Liang Gou, Wei Zhang, and Hao Yang. 2020. DySAT: Deep Neural Representation Learning on Dynamic Graphs via Self-Attention Networks. In *ACM International Conference on Web Search and Data Mining*. 519–527.
- [29] Amauri Souza, Diego Mesquita, Samuel Kaski, and Vikas Garg. 2022. Provably expressive temporal graph networks. *Advances in Neural Information Processing Systems* 35 (2022), 32257–32269.
- [30] Marshall Harvey Stone. 1937. Applications of the theory of Boolean rings to general topology. *Trans. Amer. Math. Soc.* 41, 3 (1937), 375–481.
- [31] Junwei Su and Shan Wu. 2025. Temporal-Aware Evaluation and Learning for Temporal Graph Neural Networks. In *The AAAI Conference on Artificial Intelligence*, Vol. 39. 20628–20636.
- [32] Junwei Su, Difan Zou, and Chuan Wu. 2024. PRES: Toward Scalable Memory-Based Dynamic Graph Neural Networks. In *International Conference on Learning Representations*.
- [33] Komal Teru, Etienne Denis, and Will Hamilton. 2020. Inductive relation prediction by subgraph reasoning. In *International Conference on Machine Learning*. PMLR, 9448–9457.
- [34] Yuxing Tian, Yiyang Qi, and Fan Guo. 2024. Freedyg: Frequency enhanced continuous-time dynamic graph model for link prediction. In *International Conference on Learning Representations*.
- [35] Katherine Tieu, Dongqi Fu, Zihao Li, Ross Maciejewski, and Jingrui He. 2025. Learnable Spatial-Temporal Positional Encoding for Link Prediction. In *International Conference on Machine Learning*.
- [36] Ilya O Tolstikhin, Neil Houlsby, Alexander Kolesnikov, Lucas Beyer, Xiaohua Zhai, Thomas Unterthiner, Jessica Yung, Andreas Steiner, Daniel Keysers, Jakob Uszkoreit, et al. 2021. Mlp-mixer: An all-mlp architecture for vision. *Advances in Neural Information Processing Systems* 34 (2021), 24261–24272.
- [37] Rakshit Trivedi, Mehrdad Farajtabar, Prasenjeet Biswal, and Hongyuan Zha. 2019. DyRep: Learning Representations over Dynamic Graphs. In *International Conference on Learning Representations*.
- [38] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in Neural Information Processing Systems* 30 (2017).
- [39] Jianwei Wang, Kai Wang, Xuemin Lin, Wenjie Zhang, and Ying Zhang. 2024. Efficient Unsupervised Community Search with Pre-Trained Graph Transformer. *The VLDB Endowment* 17, 9 (2024), 2227–2240.

- [40] Lu Wang, Xiaofu Chang, Shuang Li, Yunfei Chu, Hui Li, Wei Zhang, Xiaofeng He, Le Song, Jingren Zhou, and Hongxia Yang. 2021. Tel: Transformer-based dynamic graph modelling via contrastive learning. *arXiv preprint arXiv:2105.07944* (2021).
- [41] Wenhao Wang, Yifan Sun, and Yi Yang. 2023. A benchmark and asymmetrical-similarity learning for practical image copy detection. In *AAAI Conference on Artificial Intelligence*, Vol. 37. 2672–2679.
- [42] Xuhong Wang, Ding Lyu, Mengjian Li, Yang Xia, Qi Yang, Xinwen Wang, Xinguang Wang, Ping Cui, Yupu Yang, Bowen Sun, and Zhenyu Guo. 2021. APAN: Asynchronous Propagation Attention Network for Real-time Temporal Graph Embedding. In *International Conference on Management of Data*. 2628–2638.
- [43] Yanbang Wang, Yen-Yu Chang, Yunyu Liu, Jure Leskovec, and Pan Li. 2021. Inductive Representation Learning in Temporal Networks via Causal Anonymous Walks. In *International Conference on Learning Representations*.
- [44] Yuxia Wu, Yuan Fang, and Lizi Liao. 2024. On the feasibility of simple transformer for dynamic graph modeling. In *The ACM Web Conference*. 870–880.
- [45] Liang Xiang, Quan Yuan, Shihuan Zhao, Li Chen, Xiatian Zhang, Qing Yang, and Jimeng Sun. 2010. Temporal recommendation on graphs via long-and short-term preference fusion. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 723–732.
- [46] Da Xu, Chuanwei Ruan, Evren Körpeoglu, Sushant Kumar, and Kannan Achan. 2020. Inductive representation learning on temporal graphs. In *International Conference on Learning Representations*.
- [47] Yuanyuan Xu, Wenjie Zhang, Xuemin Lin, and Ying Zhang. 2025. UniDyG: A Unified and Effective Representation Learning Approach for Large Dynamic Graphs. *IEEE Transactions on Knowledge and Data Engineering* (2025).
- [48] Yuanyuan Xu, Wenjie Zhang, Xiwei Xu, Binghao Li, and Ying Zhang. 2024. Scalable and Effective Temporal Graph Representation Learning With Hyperbolic Geometry. *IEEE Transactions on Neural Networks and Learning Systems* (2024).
- [49] Yuanyuan Xu, Wenjie Zhang, Ying Zhang, Xuemin Lin, and Xiwei Xu. 2026. Unlocking Multi-Modal Potentials for Dynamic Text-Attributed Graph Representation. In *Annual AAAI Conference on Artificial Intelligence*.
- [50] Y. Xu, W. Zhang, Y. Zhang, M. Orlowska, and X. Lin. 2024. TimeSGN: Scalable and Effective Temporal Graph Neural Network. In *IEEE International Conference on Data Engineering*. 3297–3310.
- [51] Yuanyuan Xu, Wenjie Zhang, Ying Zhang, Xiwei Xu, and Xuemin Lin. 2025. Fast and accurate temporal hypergraph representation for hyperedge prediction. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1*. 1727–1738.
- [52] Kun Yi, Jingru Fei, Qi Zhang, Hui He, Shufeng Hao, Defu Lian, and Wei Fan. 2024. FilterNet: Harnessing frequency filters for time series forecasting. *Advances in Neural Information Processing Systems* 37 (2024), 55115–55140.
- [53] Kun Yi, Qi Zhang, Wei Fan, Shoujin Wang, Pengyang Wang, Hui He, Ning An, Defu Lian, Longbing Cao, and Zhendong Niu. 2023. Frequency-domain mlps are more effective learners in time series forecasting. *Advances in Neural Information Processing Systems* 36 (2023), 76656–76679.
- [54] Lu Yi, Jie Peng, Yanping Zheng, Fengran Mo, Zhewei Wei, Yuhang Ye, Yue Zixuan, and Zengfeng Huang. 2025. TGB-Seq Benchmark: Challenging Temporal GNNs with Complex Sequential Dynamics. In *International Conference on Learning Representations*.
- [55] Dong Yin, Raphael Gontijo Lopes, Jon Shlens, Ekin Dogus Cubuk, and Justin Gilmer. 2019. A fourier perspective on model robustness in computer vision. *Advances in Neural Information Processing Systems* 32 (2019).
- [56] Chengxuan Ying, Tianle Cai, Shengjie Luo, Shuxin Zheng, Guolin Ke, Di He, Yanming Shen, and Tie-Yan Liu. 2021. Do transformers really perform badly for graph representation? *Advances in Neural Information Processing Systems* 34 (2021), 28877–28888.
- [57] Le Yu, Leilei Sun, Bowen Du, and Weifeng Lv. 2023. Towards better dynamic graph learning: New architecture and unified library. *Advances in Neural Information Processing Systems* 36 (2023), 67686–67700.
- [58] Guozhen Zhang, Tian Ye, Depeng Jin, and Yong Li. 2023. An Attentional Multi-scale Co-evolving Model for Dynamic Link Prediction. In *The ACM Web Conference*. ACM, 429–437.
- [59] Xin Zhang, Fei Cai, Jianming Zheng, Zhiqiang Pan, Wanyu Chen, Honghui Chen, and Chonghao Chen. 2025. Triangle Matters! TopDyG: Topology-aware Transformer for Link Prediction on Dynamic Graphs. In *The ACM on Web Conference*. 3607–3617.
- [60] Zeyang Zhang, Xin Wang, Ziwei Zhang, Haoyang Li, Zhou Qin, and Wenwu Zhu. 2022. Dynamic graph neural networks under spatio-temporal distribution shift. *Advances in Neural Information Processing Systems* 35 (2022), 6074–6089.
- [61] Jingwei Zhuo, Ziru Xu, Wei Dai, Han Zhu, Han Li, Jian Xu, and Kun Gai. 2020. Learning optimal tree models under beam search. In *International Conference on Machine Learning*. PMLR, 11650–11659.
- [62] Tao Zou, Yuhao Mao, Junchen Ye, and Bowen Du. 2024. Repeat-Aware Neighbor Sampling for Dynamic Graph Learning. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4722–4733.

A Theoretical Analysis

A.1 The Proof of Theorem 1

PROOF. Starting from the input spectrum $X(f) = \text{DFT}(X(t))$, we first apply an energy-based masking mechanism

$$\mathcal{X}_{\text{low}}(f) = \mathbb{I}(|X(f)|^2 > \tau) \odot X(f). \quad (12)$$

Thus, we obtain $\mathcal{H}_L(f) = \mathbb{I}(|X(f)|^2 > \tau) \cdot \mathbf{I}_{\mathbb{C}}$, where $\mathbf{I}_{\mathbb{C}}$ is the identity matrix.

Next, we adjust the low-pass spectrum with learnable complex weights $\mathcal{W}_{\mathbb{C}} = \mathcal{W}_r + i \cdot \mathcal{W}_i$, which serves as the frequency response of a learnable finite impulse response (FIR) filter. Thus, $\mathcal{H}_P(f) = \mathcal{W}_r + i \cdot \mathcal{W}_i$ and we obtain

$$\mathcal{X}_{\text{proj}}(f) = \mathcal{H}_P(f) \cdot \mathcal{X}_{\text{low}}(f). \quad (13)$$

This allows the SeqFilter to emphasize task-relevant bands. We achieve $X_{\text{proj}}(t) = \text{IDFT}(\mathcal{X}_{\text{proj}}(f))$.

Last, we pass $X_{\text{proj}}(t)$ by a standard 1-D convolution:

$$X_{\text{out}}(t) = \text{Conv1D}(X_{\text{proj}}(t)) = H_c * X_{\text{proj}}(t), \quad (14)$$

where H_c is the convolution kernel and $*$ denotes convolution. Taking the discrete Fourier transform (DFT), we obtain

$$X_{\text{out}}(f) = \mathcal{H}_C(f) \cdot \mathcal{X}_{\text{proj}}(f), \text{ where } \mathcal{H}_C(f) = \text{DFT}(H_c). \quad (15)$$

Thus, the Conv1D layer is equivalent to a FIR filter whose frequency response is $\mathcal{H}_C(f)$.

Based on Eqs. (12) - (15), the DFT of the time-domain structure encoder output $X_{\text{out}}(t)$ satisfies

$$\begin{aligned} X_{\text{out}}(f) &= \text{DFT}(X_{\text{out}}(t)) = \mathcal{H}_C(f) \cdot \mathcal{X}_{\text{proj}}(f) \\ &= \mathcal{H}_C(f) \cdot \mathcal{H}_P(f) \cdot \mathcal{H}_L(f) \cdot X(f) \end{aligned} \quad (16)$$

By defining the composite frequency response, we obtain

$$\mathcal{H}_{\text{FSTS}}(f) = \mathcal{H}_C(f) \cdot \mathcal{H}_P(f) \cdot \mathcal{H}_L(f). \quad (17)$$

This confirms FSTS functions as a learnable composite filter in the frequency domain, whose response is exactly $\mathcal{H}_{\text{FSTS}}(f)$. $\square$

A.2 The Proof of Theorem 2

PROOF. We first derive the Wiener-optimal filter $W_{\text{opt}}(f)$ and its corresponding minimum mean-square error (MSE), and then demonstrate that our frequency-selective temporal structure (FSTS) encoder can closely approximate $W_{\text{opt}}(f)$.

Suppose an arbitrary linear time-invariant (LTI) filter $G(f)$ is used to estimate $S(t)$ from the observation $X(t)$, we obtain

$$\hat{S}(t) = (g * X)(t), \quad G(f) = \mathcal{F}(g(t)), \quad (18)$$

where $g(t)$ is the time-domain kernel of the LTI filter and $\mathcal{F}(\cdot)$ is the Fourier transform. Because $S(t)$ and $N(t)$ are wide-sense stationary and mutually independent, the one-sided error power spectrum for an arbitrary LTI estimator $G(f)$ is

$$E(f) = |1 - G(f) \cdot H(f)|^2 \cdot S_s(f) + |G(f)|^2 \cdot S_n(f). \quad (19)$$

We obtain mean-square error (MSE): $\text{MSE}(G) = \int_0^1 E(f) df$. Given $|1 - G(f) \cdot H(f)|^2 = |G(f)|^2 \cdot |H(f)|^2 - G(f) \cdot H(f) - G^*(f) \cdot H^*(f) + 1$, we obtain

$$\begin{aligned} \text{MSE}(G) &= \int_0^1 P(f) |G(f) - \frac{S_s(f)}{P(f)} H^*(f)|^2 df + \\ &\quad \int_0^1 \left(S_s(f) - \frac{S_s^2(f)}{P(f)} |H(f)|^2 \right) df, \end{aligned} \quad (20)$$

where $P(f) := |H(f)|^2 \cdot S_s(f) + S_n(f)$. Taking the variational derivative with respect to its conjugate $G^*(f)$ and setting it to zero gives the optimal solution $W_{\text{opt}}(f)$:

$$W_{\text{opt}}(f) = \frac{H^*(f) \cdot S_s(f)}{|H(f)|^2 \cdot S_s(f) + S_n(f)} = \frac{H^*(f) \cdot S_s(f)}{P(f)}. \quad (21)$$

Substituting Eq. (21) into Eq. (20) gives

$$\text{MSE}_{\min} = \int_0^1 \frac{S_s(f) \cdot S_n(f)}{P(f)} df. \quad (22)$$

Based on Eqs. (20) and (22), we obtain

$$\text{MSE}(G) = \text{MSE}_{\min} + \int_0^1 |G(f) - W_{\text{opt}}(f)|^2 \cdot P(f) df. \quad (23)$$

Eq. (23) decomposes the MSE of any filter into the unavoidable minimum error $\text{MSE}_{\min}$ and a penalty $\int_0^1 |G(f) - W_{\text{opt}}(f)|^2 \cdot P(f) df$ that measures how far $G(f)$ deviates from the Wiener solution.

Next, we demonstrate that our frequency-selective temporal structure (FSTS) encoder can approximate $W_{\text{opt}}(f)$. According to Theorem 1, our FSTS encoder is a cascade of three learnable filters

$$\mathcal{H}_{\text{FSTS}}(f) = \mathcal{H}_C(f) \cdot \mathcal{H}_P(f) \cdot \mathcal{H}_L(f), \quad (24)$$

where $\mathcal{H}_L(f)$ and $\mathcal{H}_P(f)$ are bounded, continuous, and nowhere-zero according to Eqs. (6) and (7). Because each convolution kernel in Eq. (8) can be written as a trigonometric polynomial

$$\mathcal{H}_C^{(L)}(f) = \sum_{k=0}^{L-1} H_C^k \cdot e^{-i2\pi k f}. \quad (25)$$

Thus, the set $\mathcal{T} := \{\mathcal{H}_C^{(L)} \mid L \in \mathbb{N}, H_C^k \in \mathbb{C}\}$ is a complex sub-algebra of $C(\mathbb{T})$, where $\mathbb{N}$ denotes the set of natural numbers, $\mathbb{C}$ denotes the set of all complex numbers, and $\mathbb{T}$ is the unit circle. It contains the constants and separates points on the unit circle $\mathbb{T}$. By the complex Stone–Weierstrass theorem [30], $\mathcal{T}$ is uniformly dense in $C(\mathbb{T})$, and thus also in $C([0, 1])$. Because the Wiener-optimal transfer function $W_{\text{opt}}(f)$ is continuous on the compact interval $[0, 1]$, for every $\varepsilon > 0$ there exists an $\mathcal{H}_C^{(L)}$ such that

$$\sup_{f \in [0, 1]} |\mathcal{H}_C^{(L)}(f) - W_{\text{opt}}(f)| < \varepsilon, \quad (26)$$

where $|\cdot|$ denotes the magnitude of a complex number. This gives the required uniform ε -approximation of $W_{\text{opt}}(f)$ by the FSTS convolution block.

Let $Q(f) := \mathcal{H}_P(f) \cdot \mathcal{H}_L(f)$. Since $\mathcal{H}_P$ and $\mathcal{H}_L$ are continuous, bounded, and nowhere-zero, so is Q . Hence, there exist constants m and M such that $0 < m \leq |Q(f)| \leq M < \infty$. Because $W_{\text{opt}}(f)/Q(f) \in C([0, 1])$, the Stone–Weierstrass approximation guarantees that, for any $\varepsilon > 0$, we can pick an $\mathcal{H}_C^{(L)}(f)$ such that

$$\sup_{f \in [0, 1]} \left| \mathcal{H}_C^{(L)}(f) - \frac{W_{\text{opt}}(f)}{Q(f)} \right| < \frac{\varepsilon}{M}. \quad (27)$$

Define the full transfer function $\mathcal{H}_{\text{FSTS}}(f) = \mathcal{H}_C^{(L)}(f) \cdot Q(f)$. Then, for every f ,

$$|\mathcal{H}_{\text{FSTS}}(f) - W_{\text{opt}}(f)| = |Q(f)| \cdot \left| \mathcal{H}_C^{(L)}(f) - \frac{W_{\text{opt}}(f)}{Q(f)} \right| < M \cdot \frac{\varepsilon}{M} = \varepsilon. \quad (28)$$

Thus, we obtain

$$\sup_{f \in [0, 1]} |\mathcal{H}_{\text{FSTS}}(f) - W_{\text{opt}}(f)| < \varepsilon. \quad (29)$$

Because the coefficients of $\mathcal{H}_C^{(L)}(f)$ and the functions $\mathcal{H}_P(f)$, $\mathcal{H}_L(f)$ are learnable, the FSTS encoder can be configured to attain this uniform ε -approximation. Substituting $G = \mathcal{H}_{\text{FSTS}}(f)$ into Eq. (23) and using the uniform bound in Eq. (29), we obtain

$$\text{MSE}_{\text{FSTS}} - \text{MSE}_{\min} = \int_0^1 |\mathcal{H}_{\text{FSTS}}(f) - W_{\text{opt}}(f)|^2 \cdot P(f) df \quad (30)$$

$$< \left(\sup_{f \in [0, 1]} |\mathcal{H}_{\text{FSTS}}(f) - W_{\text{opt}}(f)| \right)^2 \cdot \int_0^1 P(f) df \quad (31)$$

$$< \varepsilon^2 \int_0^1 P(f) df. \quad (32)$$

Because $\varepsilon > 0$ is arbitrary by increasing the FIR length L , we make ε as small as desired. Thus, the FSTS encoder's mean-square error can be brought arbitrarily close to the Wiener minimum. $\square$

B Additional Experiments

B.1 Datasets

We collect 8 datasets for performance evaluation from the TGB-Seq benchmark [54], where temporal graphs are naturally characterized by complex sequential dynamics. These datasets were originally collected from diverse real-world applications, including MovieLens, Taobao, Yelp, Google Maps, Flickr, YouTube, U.S. patent citations, and Wikipedia [2, 5, 14, 19, 24, 61].

B.2 Baselines

We compare our SeqFilter against 11 T-GNNs for the future temporal link prediction task. We introduce them below.

- **JODIE** leverages recurrent neural networks to update node memories and then map node memories into future representation trajectories of nodes.
- **DyRep** regards each temporal event as a temporal point process and constructs node memory update and evolving structure aggregation for temporal representation.
- **TGAT** designs a time encoding function and applies the self-attention mechanism for temporal and structural modeling to generate node representations.
- **TGN** is a unified and general learning framework for temporal graph neural networks that includes node memory modules and temporal structure learning modules.
- **CAWN** designs causal anonymous walks to capture structure evolution and employs an attention mechanism to learn these sampled walks for downstream tasks.
- **TCL** samples neighbors via Breadth-First Search (BFS) and then presents a graph transformer [56] to learn node representations on temporal graphs.
- **GraphMixer** designs a simple network architecture for temporal graph learning and leverages the MLP Mixer [36] based on historical neighbors for node representations.
- **DyGFormer** proposes a neighbor co-occurrence encoding to capture neighbor correlations and integrates them into a transformer [38] to generate node representations.

Table 4: AUC (%) of twelve T-GNNs on eight TGB-Seq datasets. The first and second place rankings are highlighted accordingly.

Datasets	ML-20M	Taobao	Yelp	GoogleLocal	Flickr	YouTube	Patent	WikiLink	Avg
JODIE	79.30 $\pm$ 1.27	89.52 $\pm$ 0.28	97.02 $\pm$ 0.33	84.45 $\pm$ 0.75	91.27 $\pm$ 2.09	88.82 $\pm$ 0.27	85.75 $\pm$ 0.21	87.81 $\pm$ 0.38	87.99
DyRep	72.91 $\pm$ 3.10	83.92 $\pm$ 0.22	93.31 $\pm$ 0.90	79.62 $\pm$ 0.58	90.94 $\pm$ 1.44	86.35 $\pm$ 0.30	84.72 $\pm$ 0.67	85.59 $\pm$ 0.32	84.67
TGAT	51.87 $\pm$ 0.96	70.26 $\pm$ 0.48	64.39 $\pm$ 0.88	66.81 $\pm$ 0.98	75.89 $\pm$ 0.29	60.05 $\pm$ 0.42	58.29 $\pm$ 0.88	60.25 $\pm$ 0.01	63.48
TGN	83.70 $\pm$ 0.74	88.52 $\pm$ 0.74	95.05 $\pm$ 0.67	76.95 $\pm$ 0.62	94.93 $\pm$ 0.13	88.86 $\pm$ 0.56	84.55 $\pm$ 0.82	TLE	87.51
CAWN	54.31 $\pm$ 0.15	83.88 $\pm$ 0.82	75.39 $\pm$ 0.45	72.89 $\pm$ 0.04	87.07 $\pm$ 0.67	85.74 $\pm$ 1.14	67.37 $\pm$ 0.48	69.85 $\pm$ 0.20	74.56
TCL	51.87 $\pm$ 2.24	45.95 $\pm$ 6.66	58.59 $\pm$ 11.04	69.94 $\pm$ 3.29	69.86 $\pm$ 16.44	70.62 $\pm$ 14.29	52.80 $\pm$ 0.68	51.19 $\pm$ 0.53	58.85
GraphMixer	81.27 $\pm$ 0.49	84.90 $\pm$ 1.53	87.46 $\pm$ 0.58	73.87 $\pm$ 0.13	93.54 $\pm$ 0.02	91.27 $\pm$ 0.12	84.16 $\pm$ 0.17	88.38 $\pm$ 0.29	85.61
DyGFormer	62.34 $\pm$ 0.06	86.13 $\pm$ 0.71	83.12 $\pm$ 0.77	72.83 $\pm$ 0.12	93.73 $\pm$ 0.02	86.65 $\pm$ 1.28	50.63 $\pm$ 1.40	TLE	76.49
CNEN	82.72 $\pm$ 0.15	90.73 $\pm$ 0.01	94.08 $\pm$ 0.06	90.95 $\pm$ 0.18	97.19 $\pm$ 0.01	90.90 $\pm$ 0.16	81.61 $\pm$ 0.14	95.62 $\pm$ 0.11	90.47
TPNet	82.68 $\pm$ 0.07	87.49 $\pm$ 0.04	88.19 $\pm$ 0.01	74.48 $\pm$ 0.33	96.15 $\pm$ 0.01	93.54 $\pm$ 0.17	83.99 $\pm$ 0.38	91.06 $\pm$ 0.13	87.19
FreeDyG	83.45 $\pm$ 0.68	86.55 $\pm$ 0.01	88.47 $\pm$ 0.32	73.62 $\pm$ 0.25	95.75 $\pm$ 0.07	91.46 $\pm$ 0.13	48.97 $\pm$ 2.55	95.55 $\pm$ 0.03	82.98
SeqFilter	96.09 $\pm$ 0.18	96.27 $\pm$ 0.05	98.16 $\pm$ 0.01	88.63 $\pm$ 0.02	98.84 $\pm$ 0.16	98.02 $\pm$ 0.01	92.16 $\pm$ 0.30	95.14 $\pm$ 0.11	95.41

Table 5: Performance comparison of 12 T-GNNs on general temporal graphs in MRR (%) on future link prediction. Results marked with an asterisk are collected from [54].

Datasets	Wikipedia	Reddit	Average
JODIE*	76.94 $\pm$ 0.28	77.92 $\pm$ 0.10	77.43
DyRep*	68.09 $\pm$ 1.45	75.30 $\pm$ 0.30	71.69
TGAT*	72.42 $\pm$ 0.38	76.69 $\pm$ 0.52	74.55
TGN*	81.16 $\pm$ 0.19	79.82 $\pm$ 0.26	80.49
CAWN*	88.23 $\pm$ 0.33	87.31 $\pm$ 0.32	87.77
TCL*	45.47 $\pm$ 3.48	36.09 $\pm$ 2.10	40.78
GraphMixer*	72.14 $\pm$ 0.80	71.73 $\pm$ 0.32	71.93
DyGFormer*	84.64 $\pm$ 0.43	83.57 $\pm$ 1.42	84.10
CNEN	82.91 $\pm$ 0.19	88.32 $\pm$ 0.30	85.62
TPNet	65.48 $\pm$ 0.71	50.38 $\pm$ 0.26	57.93
FreeDyG	76.62 $\pm$ 0.41	29.97 $\pm$ 0.92	53.30
SeqFilter	91.74 $\pm$ 0.23	95.60 $\pm$ 0.12	93.67

- **CNEN** designs a hashtable-based memory to identify co-neighbors, generating both long-term and short-term structural encodings for node representations.
- **TPNet** unifies existing relative encodings as a function of temporal walk matrices and efficiently encodes pairwise information through temporal walk matrix projection.
- **FreeDyG** design a frequency-enhanced MLP-Mixer to capture periodic temporal patterns, together with a neighbor encoding scheme for temporal link prediction.

B.3 Experimental Configuration

We conduct all experiments on a single machine equipped with Intel(R) Xeon(R) Gold 6240R CPU @ 2.40GHz CPUs and NVIDIA A100 with 40 GB VRAM. We implement SeqFilter in alignment with DyGLib and the TGB-Seq benchmark. For fair comparison, we set a batch size of 600, 20 sampled neighbors, and a learning rate of 0.001.

Rhythm memory employs a convolution kernel of size 4, while the frequency-selective structure encoder sets a convolution kernel size of 8. We configure the threshold τ as 0.1 for GoogleLocal and Patent, 0.5 for Taobao, and 0.2 for the remaining datasets. Following the TGB-Seq benchmark, each dataset is split chronologically into 70% training, 15% validation, and 15% test sets.

For the eight baselines (JODIE, DyRep, TGAT, TGN, CAWN, TCL, GraphMixer, and DyGFormer), we adopt the hyperparameter settings provided by the TGB-Seq benchmark. The remaining three baselines use the configurations specified in their original papers.

B.4 Effectiveness Evaluation in Binary Setting

We report the results of future link prediction in the binary classification in AUC across eight TGB-Seq datasets.

Obs-10: Effectiveness in the binary setting. In Table 4, the proposed SeqFilter outperforms 11 baselines by an average of 3.60% in AUC, demonstrating its effectiveness in relatively simple prediction settings. Additionally, several baselines like GraphMixer, CNEN and TPNet perform respectably in binary classification, but they fall short in ranking settings, underscoring their limited generalization.

B.5 Evaluation on General Temporal Graphs

We assess the performance of the proposed SeqFilter on two general temporal graphs (Wikipedia and Reddit) collected from [27, 57], where both temporal graphs contain numerous repeated edges. We use the future link prediction task regarding MRR and report the results in Table 5. Note that we use the same data split as the TGB-Seq benchmark for the training/validation/test split.

Obs-11: Effectiveness on general temporal graphs. Compared to the best competitor, SeqFilter achieves an absolute improvement of 3.51% on Wikipedia and 7.28% on Reddit in MRR, highlighting its superior capability to capture underlying temporal and structural patterns on general temporal graphs, even in the presence of repeated edges. These results underscore the broad applicability of SeqFilter to a wide range of temporal graph applications.